

Software Engineering

Dr. Md. Nasim Adnan

- He had the classic look of a senior executive for a major software company—mid-40s, slightly graying at the temples, trim and athletic, with eyes that penetrated the listener as he spoke.
- But what he said shocked me. “Software is *dead*.”
- I blinked with surprise and then smiled. “You’re joking, right? The world is driven by software and your company has profited handsomely because of it. It isn’t dead! It’s alive and growing.”
- He shook his head emphatically. “No, it’s dead . . . at least as we once knew it.”
- I leaned forward. “Go on.”

- He spoke while tapping the table for emphasis. “The old-school view of software—you buy it, you own it, and it’s your job to manage it—that’s coming to an end.
- Today, with Web 2.0 and pervasive computing coming on strong, we’re going to be seeing a completely different generation of software. It’ll be delivered via the Internet and will look exactly like it’s residing on each user’s computing device . . . but it’ll reside on a far-away server.”
- Is software *really dead? If it was, you wouldn’t be reading this book!*

- Computer software continues to be the single most important technology on the world stage. And it's also a prime example of the law of unintended consequences.
- Fifty years ago no one could have predicted that software would become an indispensable technology for business, science, and engineering.
- When Dijkstra married Maria in 1957, he was required as a part of the marriage rites to state his profession. He stated that he was a Programmer, which was unacceptable to the authorities, there being no such profession then in The Netherlands.

- No one could predict that millions of computer programs would have to be corrected, adapted, and enhanced as time passed.
- The burden of performing these “maintenance” activities would absorb more people and more resources than all work applied to the creation of new software.
- As software’s importance has grown, the software community has continually attempted to develop technologies that will make it easier, faster, and less expensive to build and maintain high-quality computer programs.

The Nature of Software

- Today, software takes on a dual role. It is a product, and at the same time, the vehicle for delivering a product. As a product, it delivers the computing potential embodied by computer hardware or more broadly, by a network of computers that are accessible by local hardware.
- Whether it resides within a mobile phone or operates inside a mainframe computer, software is an information transformer — producing, managing, acquiring, modifying, displaying, or transmitting information that can be as simple as a single bit or as complex as a multimedia presentation derived from data acquired from dozens of independent sources.

- As the vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs (software tools and environments).
- Software delivers the most important product of our time — *information*. It transforms personal data (e.g., an individual's financial transactions) so that the data can be more useful in a local context; it manages business information to enhance competitiveness; it provides a gateway to worldwide information networks (e.g., the Internet), and provides the means for acquiring information in all of its forms.

- Today, a huge software industry has become a dominant factor in the economies of the industrialized world. Teams of software specialists, each focusing on one part of the technology required to deliver a complex application, have replaced the lone programmer of an earlier era.
- And yet, the questions that were asked of the lone programmer are the same questions that are asked when modern computer-based systems are built:

- Why does it take so long to get software finished?
- Why are development costs so high?
- Why can't we find all errors before we give the software to our customers?
- Why do we spend so much time and effort maintaining existing programs?
- Why do we continue to have difficulty in measuring progress as software is being developed and maintained?



- Margaret Hamilton in 1969, standing next to listings of the software she and her MIT team produced for the Apollo project.

Defining Software

- **Software is:**
 - (1) instructions (computer programs) that when executed provide desired features, function, and performance;
 - (2) data structures that enable the programs to adequately manipulate information, and
 - (3) descriptive information in both hard copy and virtual forms that describes the operation and use of the programs.

- Software has characteristics that are considerably different than those of hardware:
 - *Software is developed or engineered; it is not manufactured in the classical sense*
 - *Software doesn't "wear out."*
 - *Although the industry is moving toward component-based construction, most software continues to be custom built.*

- The next Figure depicts failure rate as a function of time for hardware. The relationship, often called the “bathtub curve,”.

Early Life (High Failure Rate at the Start):

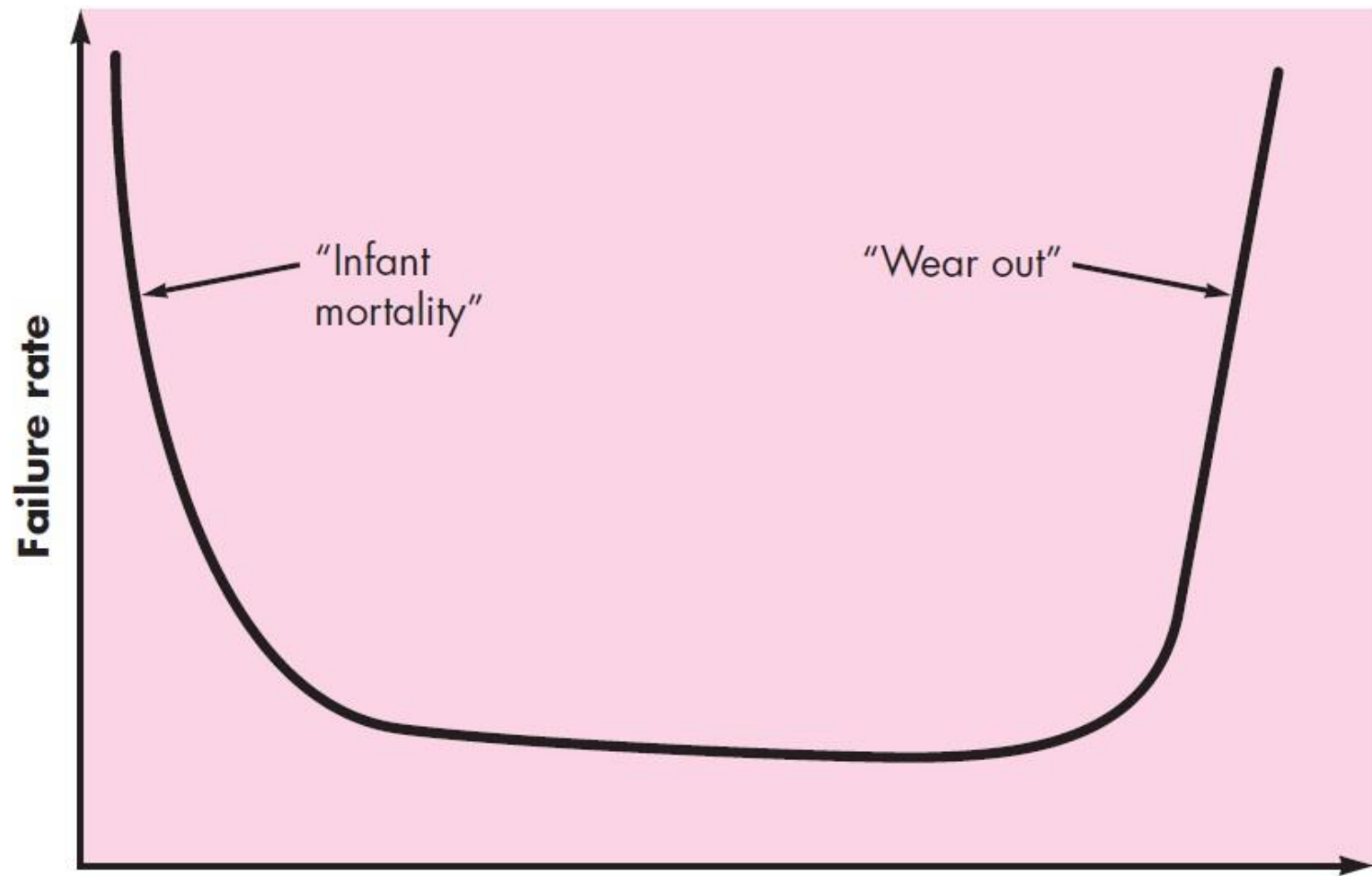
When you first get a piece of hardware (like a computer, phone, etc.), it might break or malfunction because of problems from the factory or design flaws. These failures happen early in the product's life. However, as manufacturers fix these issues, the product becomes more reliable.

Middle Life (Low Failure Rate):

After fixing any early problems, the hardware works well and doesn't break easily. It stays reliable for a good period, and you don't have to worry about failures. This is the best phase of the product's life.

Later Life (Failure Rate Rises Again):

Over time, the hardware starts to experience wear and tear. Things like dust, vibrations, extreme temperatures, or accidental damage can cause it to break down. This makes the failure rate rise again as the product gets older.



Software Doesn't Wear Out Like Hardware:

Unlike hardware, which wears out over time because of things like dust, heat, and physical wear, software doesn't experience this kind of physical deterioration. So, you wouldn't expect software to fail just because it's getting old.

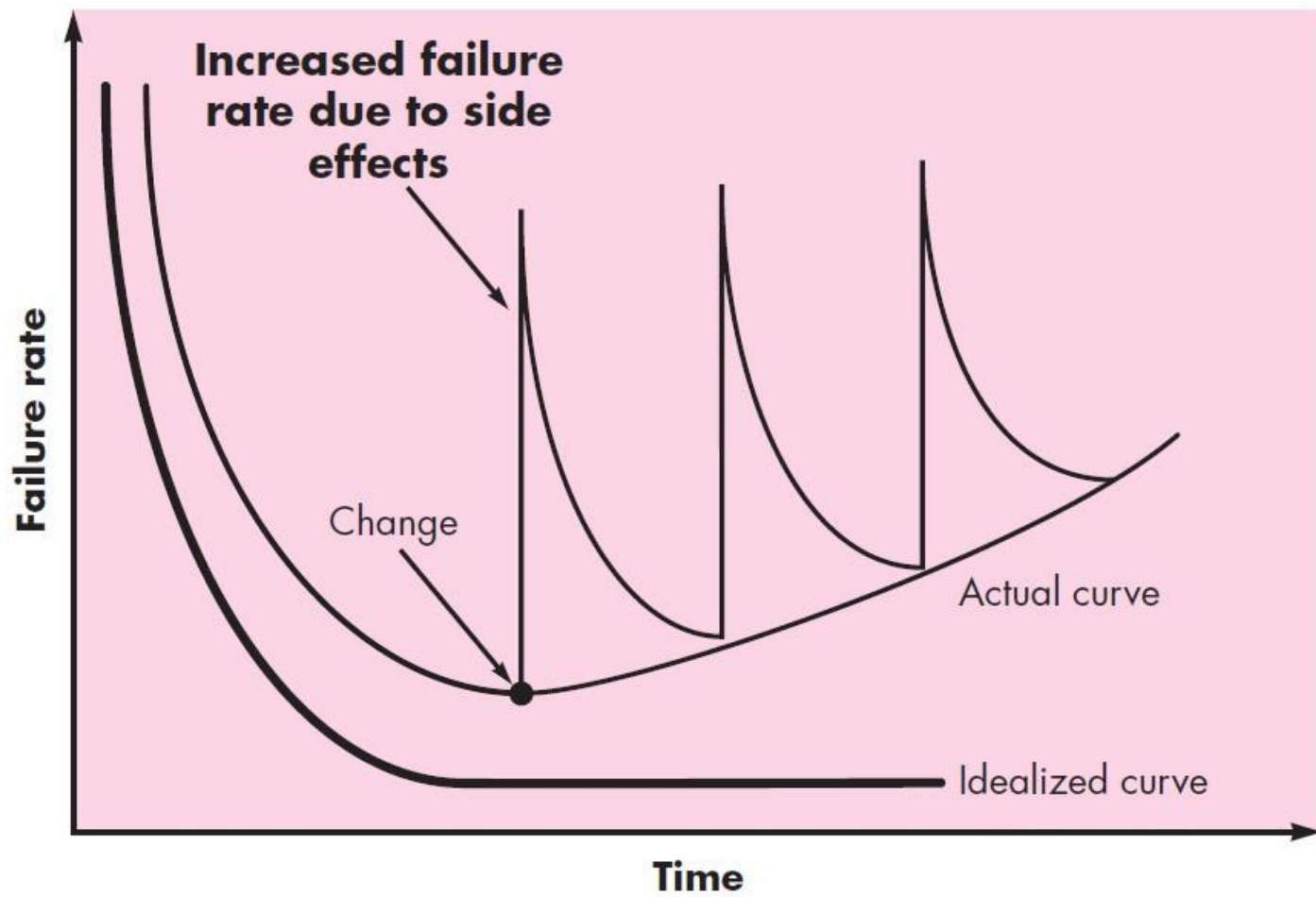
Idealized Curve (Theoretical, Simple View):

In theory, software should follow a simple failure rate curve:

Early Life (High Failure Rate): When software is first developed, there are usually bugs or defects that cause it to fail or crash. This is why the failure rate is high at the beginning.

Middle Life (Low Failure Rate): As developers find and fix these bugs, the software stabilizes, and the failure rate drops to a steady, low level.

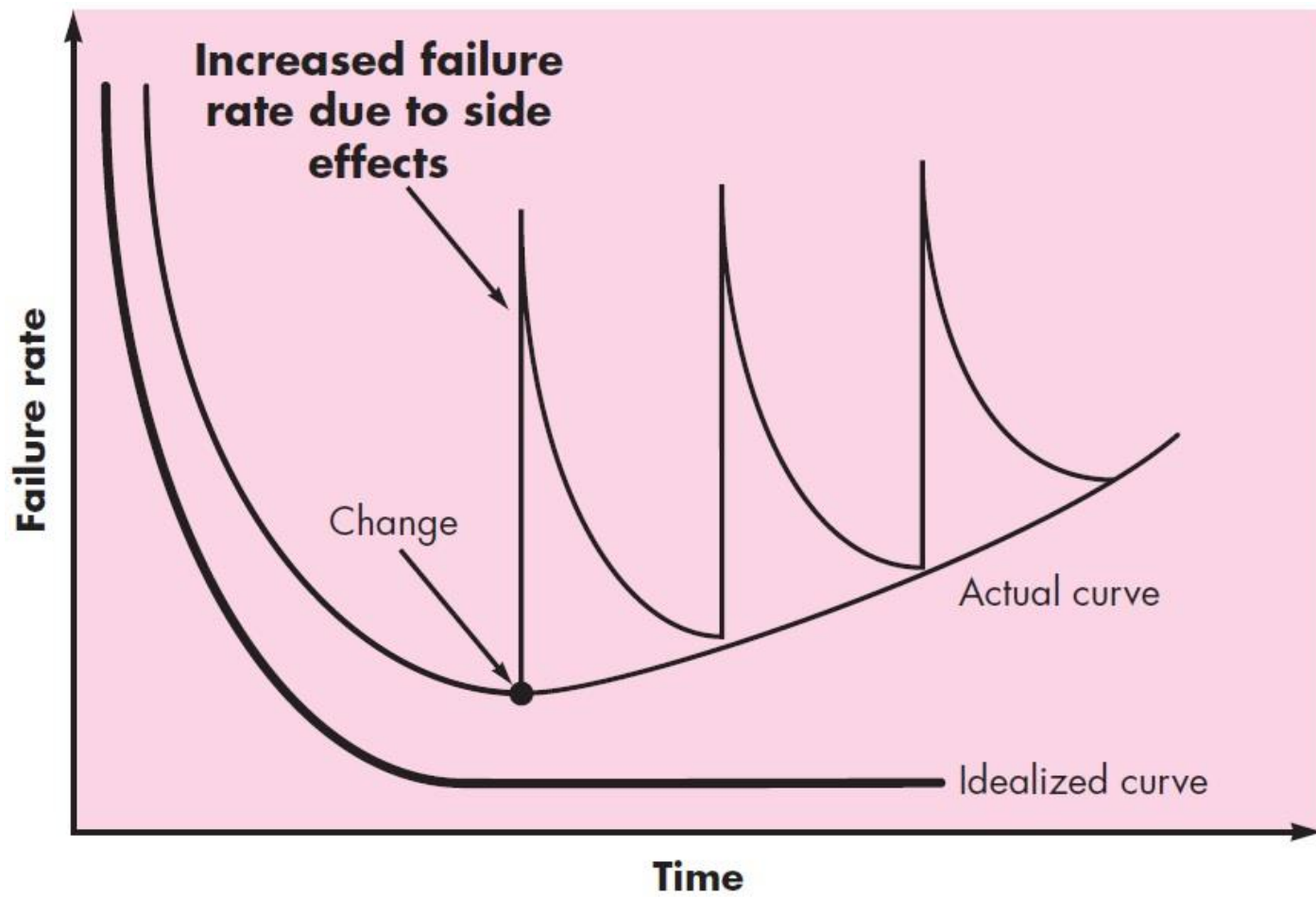
Later Life (Steady or No Wear-Out): Since software doesn't physically wear out, it would theoretically just keep working fine without getting worse.



Real-World Failure Curve (Actual Curve):

The idealized curve oversimplifies how software actually behaves. In the real world:

- **As developers make changes to the software (to add new features or fix bugs), new problems or errors might get introduced. This causes the failure rate to spike (go up) again.
- ** Once these new problems are fixed, the failure rate goes back down to a stable level.
- ** However, as more changes are made, it's possible that the minimum failure rate starts to rise over time. This happens because each change, even if it fixes some issues, might introduce new problems, making the software slowly deteriorate or become less stable.



Software Application Domains

- System Software
- Application Software
- Engineering/Scientific Software
- Embedded Software
- Product-line Software
- Web Applications
- Artificial Intelligence Software

"Software application domains" refer to the different areas or industries where software is applied to solve problems or provide services.

Software Engineering

- In order to build software that is ready to meet the challenges of the twenty-first century, you must recognize a few simple realities:

-First, it's very important to clearly understand the problem before starting to build the software. If we don't understand the problem well, the software might not work properly.

-Second, the needs of people, businesses, and governments are getting more complicated every year like with income tax systems, for example.

Because of this growing complexity, big teams are now needed to build software that used to be made by just one person. That's why planning and designing the software carefully has become a very important part of the process.

- *The hardest thing in the world to understand is the income tax.* – Albert Einstein

- People, businesses, and governments now depend more and more on software to make important decisions and run their daily work. If the software doesn't work properly, it can cause small problems — or even very serious failures. That's why software needs to be high quality.
- Also, when an application becomes more useful and valuable, more people will start using it, and it will likely be used for a long time. As this happens, people will want new features and updates. So, the software should be easy to update and improve — in other words, it should be maintainable.

Footnote

Footnote – Types of Information Systems:

DSS (Decision Support System)

Helps in long-term planning and making important decisions.

Example: Planning changes in staff or company structure.

MIS (Management Information System)

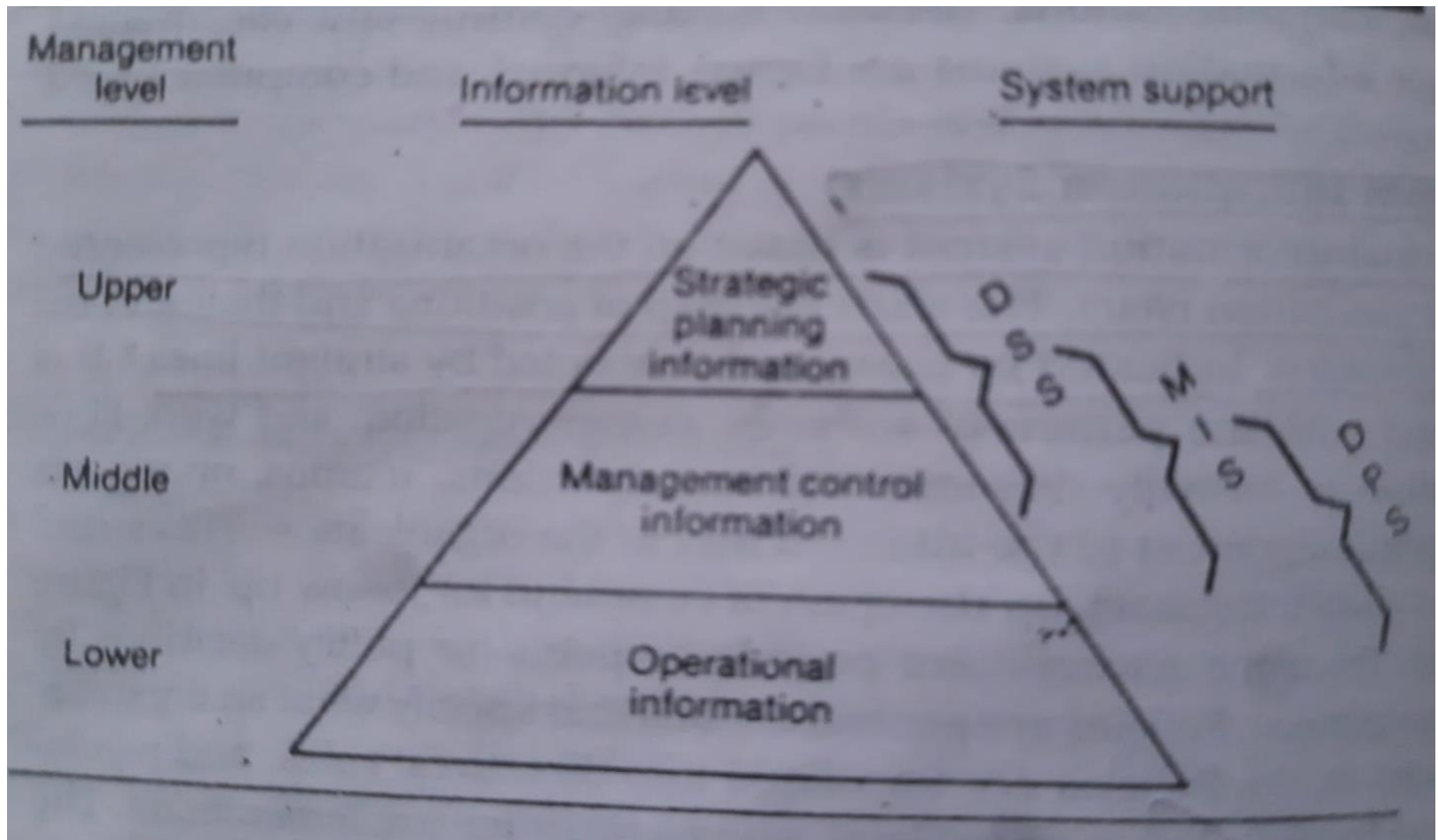
Most people are familiar with this.

Used for analyzing business performance, like monthly or yearly sales reports.

DPS (Data Processing System)

Enforce day to day rules and regulations of a business

Examples: Employee absence sheet, current stock of products



Focus Area: MIS

- A key element of MIS is the database. MIS cannot be implemented effectively without the help of a database
- A database is a nonredundant collection of interrelated data items that can be accessed through application programs and available to many users

Decision Making

❖ Decision Making – Elements:

Data/Information – Needed to understand the problem and options.

Problem/Goal – The issue to solve or the goal to reach.

Alternatives – Different choices or actions to consider.

Analysis – Comparing the options using facts and tools.

Choice – Picking the best option.

Implementation – Putting the chosen option into action.

Review – Checking if the decision worked well or needs change.

Decision Making

MIS (Management Information System)

Collects and organizes everyday business transactions.

Examples:

Production MIS – handles production data

Personnel MIS – handles employee data

The data is grouped and analyzed based on different dimensions, like time, department, or product.

DSS (Decision Support System)

In simple words, a DSS uses results from MIS systems along with external data (like market trends) to help managers make better decisions

All of these facts point to one clear idea:

No matter what type of software we build or where it is used, it should be carefully planned and developed—just like in engineering. This brings us to the main topic: software engineering.

Over time, many experts have given their own definitions of software engineering. But one of the most well-known and commonly used definitions was given by Fritz Bauer:

"Software engineering is the use of proper engineering methods to create software that is affordable, dependable, and works well on real computers."

The Software Process

- *A process is a collection of activities, actions, and tasks that are performed when some work product is to be created.*

1. An activity is a big step that works toward a general goal.

Example: Talking and planning with the people who need the software (stakeholders).

This happens in all types of projects, no matter the size or complexity.

2. An action includes several smaller tasks that help create an important part of the software.

Example: Making the overall design of the system (architectural design).

3. A task is a very specific job with a clear goal.

Example: Running a unit test to check a small part of the code.

This produces something you can see or use.

In software engineering, a process is not a strict set of rules that must always be followed the same way.

Instead, it's a flexible guide that helps the software team choose the right steps based on their project.

The main goal is always to deliver the software on time and with good enough quality to make both the client and the users happy.

Process Framework

- A generic process framework for software engineering encompasses five activities:
- **Communication.** Before any technical work can commence, it is critically important to communicate and collaborate with the customer (and other stakeholders).
- The intent is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.

- **Planning.** Any complicated journey can be simplified if a map exists. A software project is a complicated journey, and the planning activity creates a “map” that helps guide the team as it makes the journey.
- This "map" is called a software project plan. It explains what technical tasks need to be done, what problems or risks might happen, what resources (like people, tools, and time) are needed, what the team is expected to deliver, and when everything should be completed. For example, like how the Padma Bridge project needed detailed planning, software projects also need careful planning to succeed.

- **Modeling.** People in many fields — like architects, engineers, or carpenters — use models or sketches before starting their work. These models help them see the overall structure, understand how different parts fit together, and plan everything clearly. It gives them a big-picture view before they start building.

In the same way, software engineers create models to understand what the software should do and how it should be built. These models start simple and can be made more detailed as needed. This helps the team clearly understand the requirements and design a solution that works.

- **Construction.** This is the stage where the actual coding happens. Developers write the program manually or use tools to help generate code. Along with writing the code, they also test it to find and fix any errors, making sure everything works as expected.
- **Deployment.** Once the software (or part of it) is ready, it is delivered to the customer. The customer then tries it out and gives feedback based on their experience. This feedback helps the team know what's working well and what might need improvement.

- As discussed previously, a generic process framework for software engineering defines five framework activities—**communication, planning, modeling, construction, and deployment.**
- We now describe the principles that focus on software engineering practice as a whole.

The First Principle: The Reason It All Exists

- A software system exists for one reason: to provide value to its users. Everything about the system should be planned with this in mind.
- Before specifying a system requirement, before noting a piece of system functionality, before determining the hardware platforms or development processes, ask yourself questions such as: “Does this add real value to the system?”
- If the answer is “no,” don’t do it. All other principles support this one.

The Second Principle: KISS (Keep It Simple, Stupid!)

- Software design should never be random or chaotic—it requires careful thought and planning. While many factors must be considered in any design effort, the guiding rule is:

Keep the design as simple as possible.

Simplicity makes the system easier to understand, develop, and maintain, which reduces the risk of errors and ensures long-term sustainability.

The Third Principle: Maintain the Vision

- Every successful software project needs a clear and shared vision from the beginning. This means everyone on the team should understand what the project is about and what it should become.

If there's no clear vision, the project can become confusing and disorganized, with different people pulling it in different directions. This often results in a system that feels like a messy mix of different designs that don't work well together—like parts that don't fit or features that break old ones (lack of backward compatibility).

Keeping a strong, consistent vision helps the whole system stay coherent, reliable, and easier to manage.

The Fourth Principle: What You Produce, Others Will Consume

- In most real-world scenarios, software systems are rarely developed or maintained in isolation. Every artifact you produce—be it specifications, designs, or source code—will likely be used, reviewed, or maintained by someone else.

Therefore, it is essential to design, implement, and document software with the understanding that others will need to comprehend, extend, debug, or support what you've created. The audience for any development product is potentially broad—ranging from end users and testers to future developers and maintainers.

- Being mindful of others throughout development not only improves team collaboration and long-term maintainability, but also adds significant value to the system.
- Making their job easier adds value to the system.

The Fifth Principle: Be Open to the Future

In a rapidly changing technology landscape —where requirements shift quickly and hardware becomes outdated in months—software must be designed with adaptability in mind. While many systems have short lifespans, true industrial-strength software is expected to last much longer.

To achieve this, software should be built in a way that welcomes future changes, whether in features, technologies, or platforms. The most successful systems are those that have been designed from the beginning to evolve easily with time, rather than resist change.

The Sixth Principle: Plan Ahead for Reuse

Reusing software components —such as code, designs, or modules—can significantly save time, reduce effort, and improve quality. However, achieving effective reuse is one of the most challenging goals in software development.

While object-oriented programming and other modern approaches promote reuse, the benefits are not automatic. To truly take advantage of reuse, developers must plan for it early — designing systems with modularity, flexibility, and general-purpose components in mind.

Successful reuse requires intentional design choices and upfront thinking, not just relying on tools or languages.

The Seventh principle: Think!

- This last rule is often ignored, but it is very important. Before doing anything — like writing code, building a system, or making a choice — take some time to think carefully. Doing this usually helps you get better results .
- When you understand what you're doing and plan ahead, you're more likely to do it correctly and avoid mistakes.

